

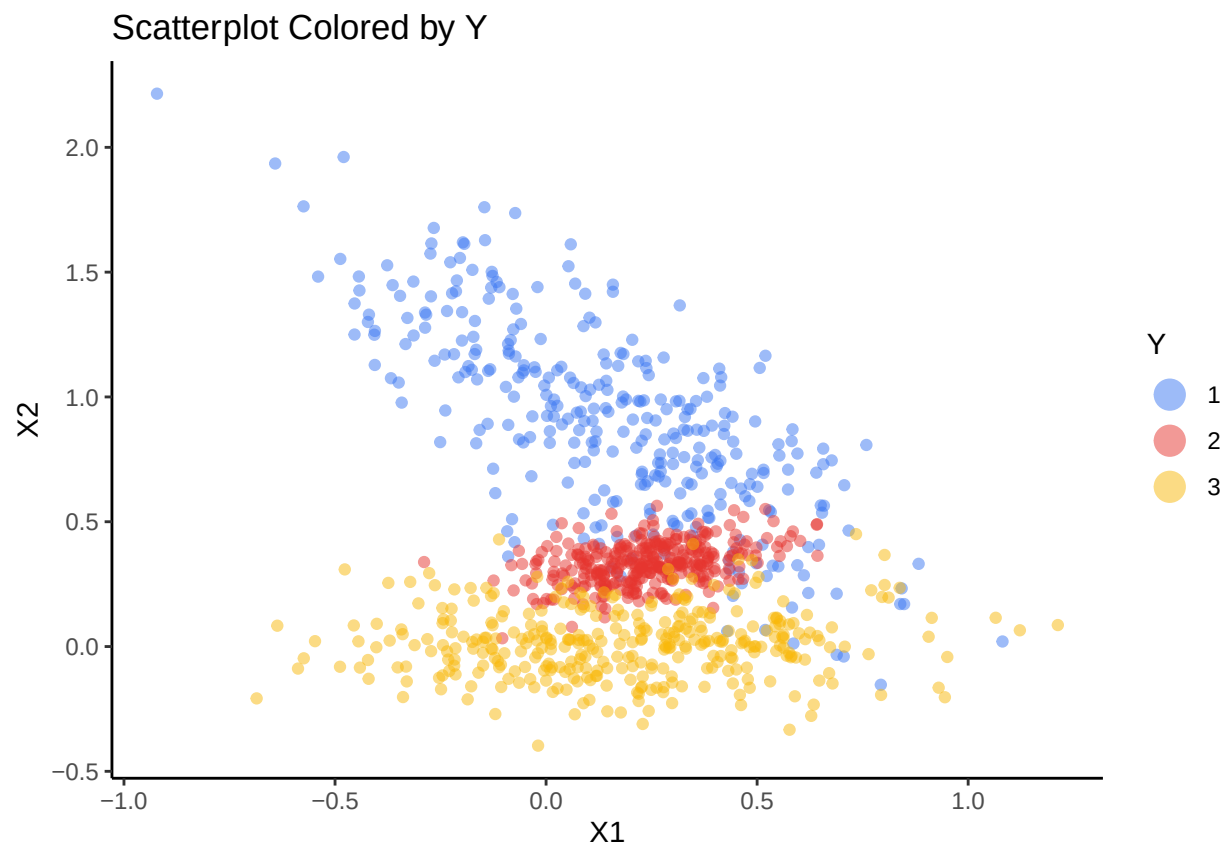
Classification with Several Cases

Emma Johnson

March 17, 2025

Problem 1

1.



2.

```
set.seed(5) # reproducibility

# Split train/test sets 50/50:
train_ind <- sample(1:nrow(data1), size = 0.5 * nrow(data1))
train <- rep(FALSE, nrow(data1))
train[train_ind] <- TRUE # training set: 500 obs
```

```
test <- data1[!train, ] # held out set (testing): 500 obs
test.class <- data1$Y[!train] # true Y values for testing set
```

3.

```
lda.fit <- lda(
  Y ~ X1 + X2,
  data = data1,
  subset = train
)
lda.fit
```

```
## Call:
## lda(Y ~ X1 + X2, data = data1, subset = train)
##
## Prior probabilities of groups:
##      1      2      3
## 0.328 0.328 0.344
##
## Group means:
##           X1           X2
## 1 0.1374246 0.92569028
## 2 0.2359942 0.33196721
## 3 0.1821332 0.01984174
##
## Coefficients of linear discriminants:
##           LD1           LD2
## X1 -1.572125 -3.5599949
## X2 -4.340209 -0.2160405
##
## Proportion of trace:
##      LD1      LD2
## 0.9949 0.0051
```

```
# Predictions using LDA model:
lda.class <- predict(lda.fit, test)$class
```

```
# Confusion matrix:
table(lda.class, test.class)
```

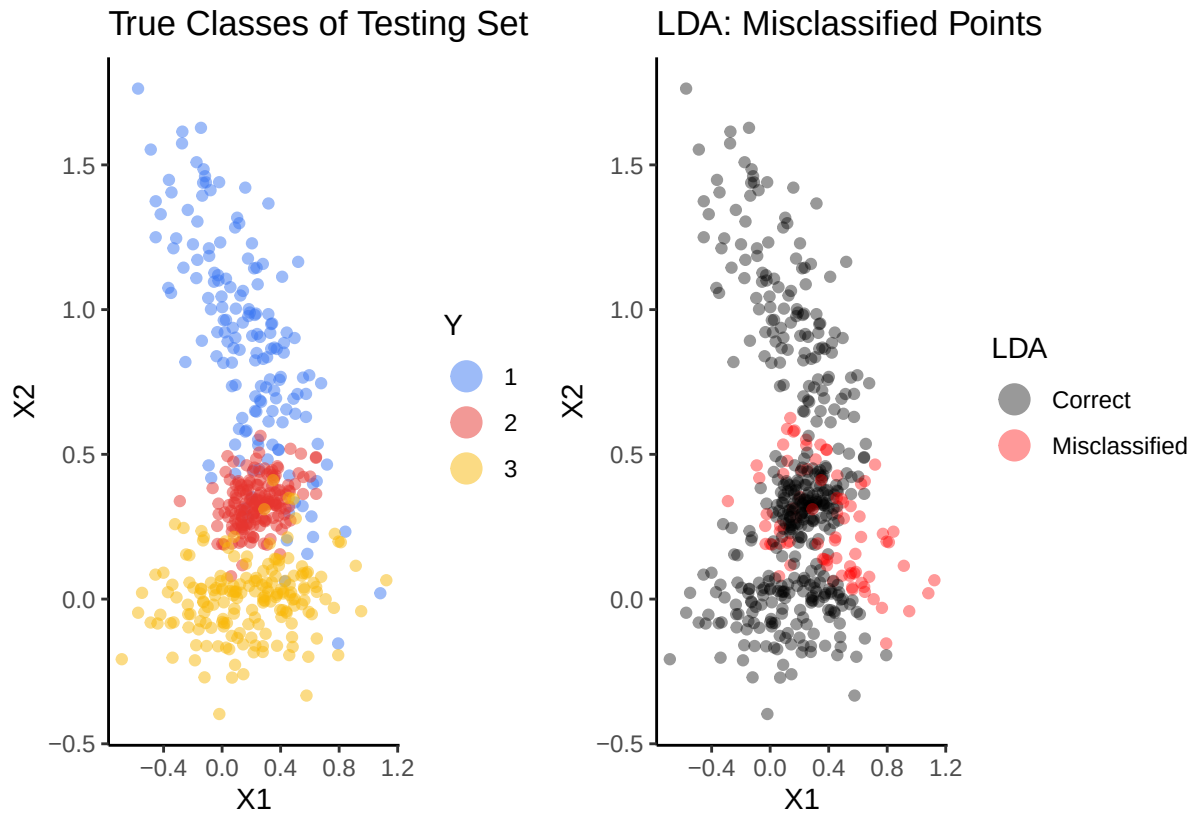
```
##           test.class
## lda.class  1    2    3
##           1 122    0    0
##           2  33 150  32
##           3   2  13 148
```

```
# Rate of correct classifications:
mean(lda.class == test.class)
```

```
## [1] 0.84
```

```
# Test error:
mean(lda.class != test.class)
```

```
## [1] 0.16
```



4.

```
qda.fit <- qda(
  Y ~ X1 + X2,
  data = data1,
  subset = train
)
qda.fit
```

```
## Call:
## qda(Y ~ X1 + X2, data = data1, subset = train)
##
## Prior probabilities of groups:
##      1      2      3
## 0.328 0.328 0.344
##
## Group means:
##           X1           X2
## 1 0.1374246 0.92569028
## 2 0.2359942 0.33196721
## 3 0.1821332 0.01984174
```

```
# Predictions using QDA model:
qda.class <- predict(qda.fit, test)$class
```

```
# Confusion matrix:
```

```
table(qda.class, test.class)
```

```
##          test.class
## qda.class  1    2    3
##          1 136    8    1
##          2  15 149    8
##          3   6   6 171
```

```
# Rate of correct classifications:
```

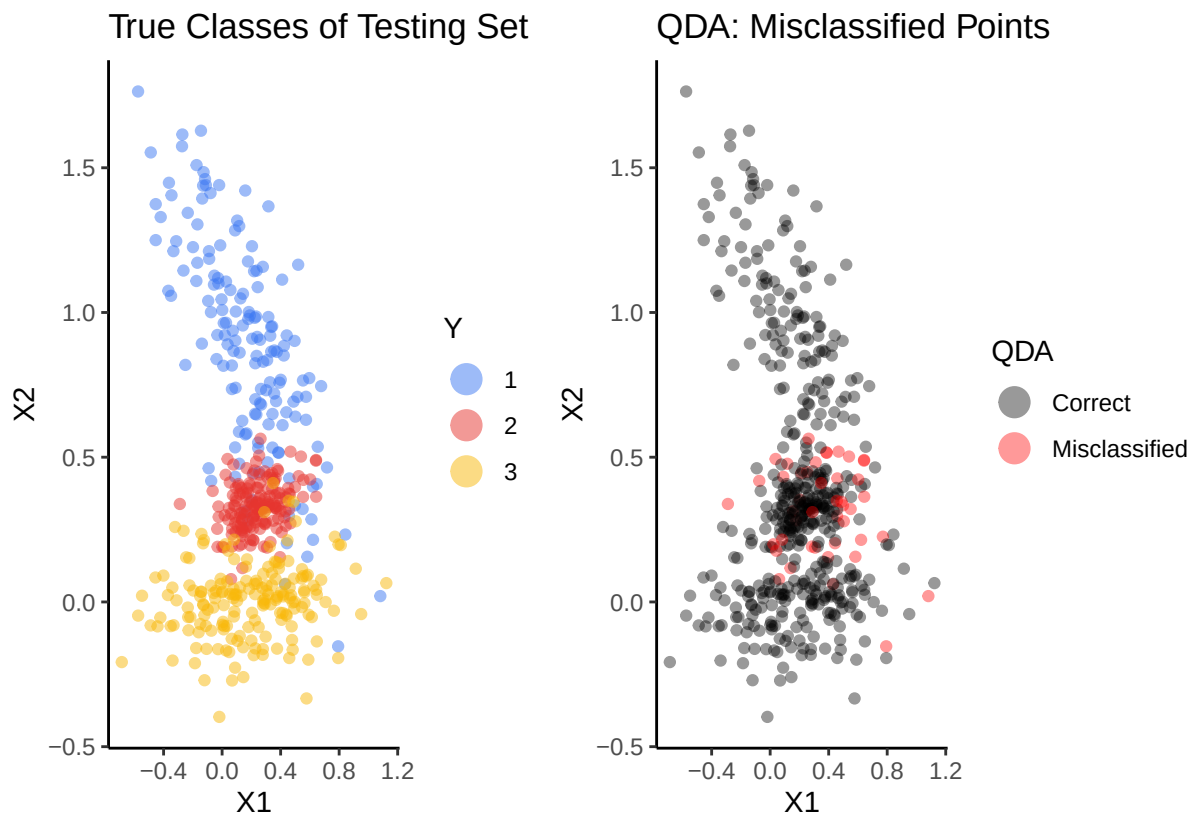
```
mean(qda.class == test.class)
```

```
## [1] 0.912
```

```
# Test error:
```

```
mean(qda.class != test.class)
```

```
## [1] 0.088
```



5.

```
# Standardizing X1 & X2:
standardized.X <- scale(data1[,c(1:2)])
```

```
# Updating training & testing sets:
```

```
train.X <- standardized.X[train, ]
```

```
test.X <- standardized.X[!train, ]
```

```
train.class <- data1$Y[train]
```

```

test.class <- data1$Y[!train]

# Predictions using KNN with k = 10:
set.seed(5)
knn.pred <- knn(train.X, test.X, train.class, k = 10)

# Confusion Matrix:
table(knn.pred, test.class)

##           test.class
## knn.pred  1    2    3
##      1 133    5    2
##      2  19 150    9
##      3   5   8 169

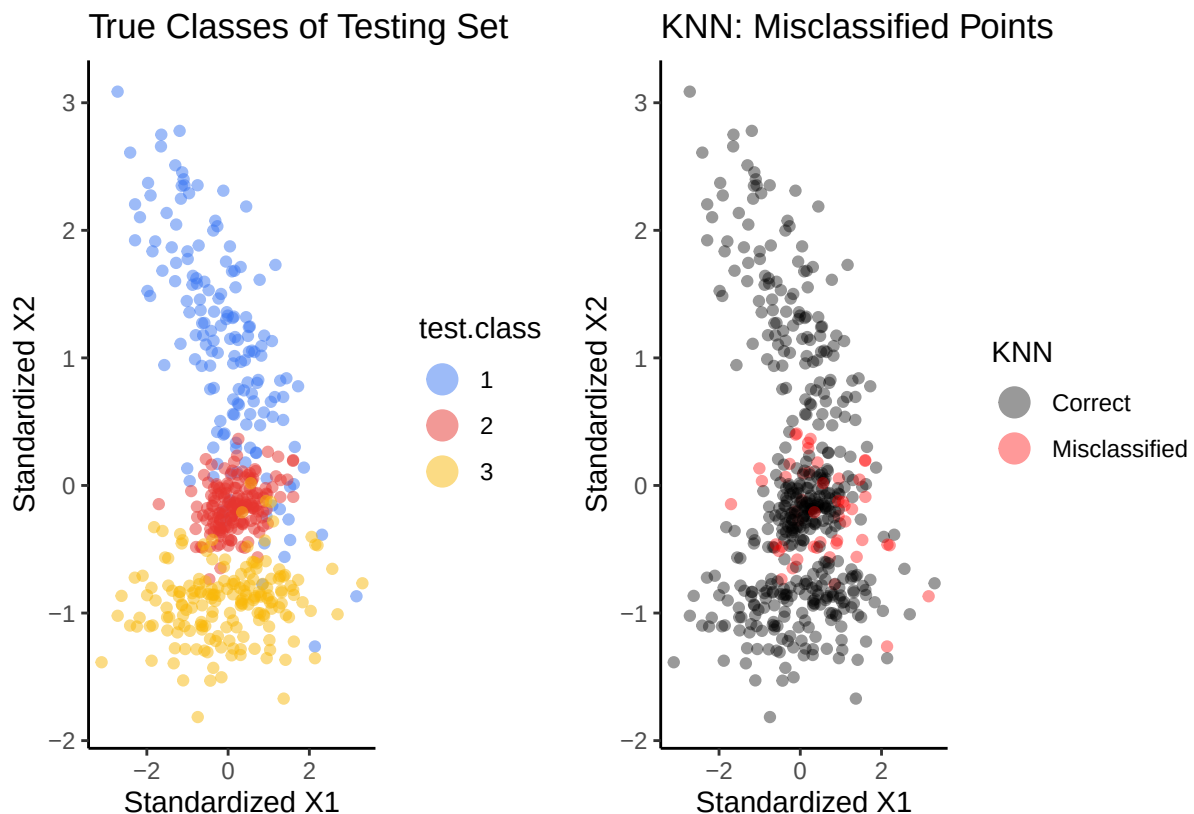
# Rate of correct classifications:
mean(knn.pred == test.class)

## [1] 0.904

# Test Error:
mean(knn.pred != test.class)

## [1] 0.096

```



6.

Comparing the three testing procedures, the results are as follows:

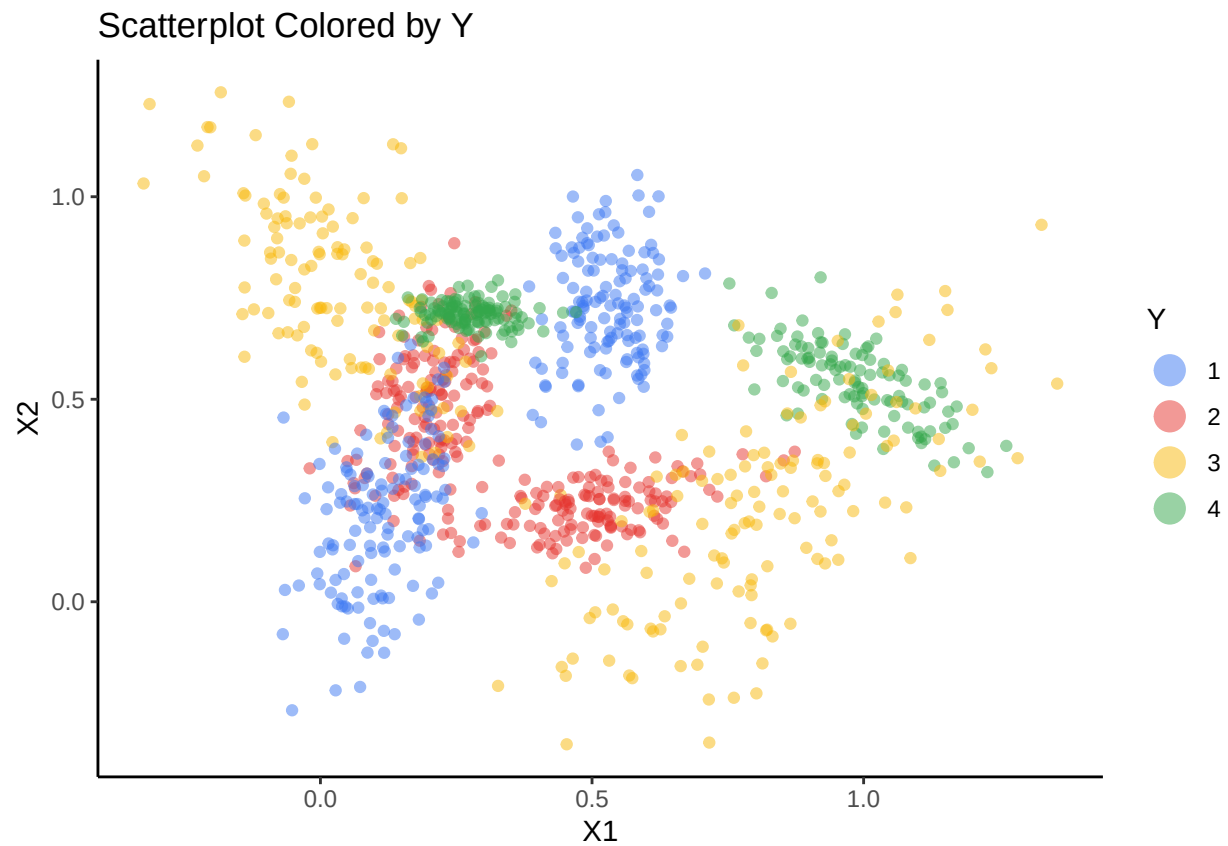
Table 1: Test Error Comparisons

Method	Test_Error
LDA	16%
QDA	8.8%
KNN	9.6%

For this data, QDA performed better than LDA and KNN.

Problem 2

1.



2.

```
set.seed(10) # reproducibility

# Split train/test sets 50/50:
train_ind <- sample(1:nrow(data2), size = 0.5 * nrow(data2))
train <- rep(FALSE, nrow(data2))
train[train_ind] <- TRUE # training set: 500 obs

test <- data2[!train, ] # held out set (testing): 500 obs
test.class <- data2$Y[!train] # true Y values for testing set
```

3.

```
qda.fit <- qda(  
  Y ~ X1 + X2,  
  data = data2,  
  subset = train  
)  
qda.fit
```

```
## Call:  
## qda(Y ~ X1 + X2, data = data2, subset = train)  
##  
## Prior probabilities of groups:  
##      1      2      3      4  
## 0.276 0.256 0.266 0.202  
##  
## Group means:  
##           X1           X2  
## 1 0.3290375 0.4882679  
## 2 0.3507495 0.3612605  
## 3 0.4526473 0.4717941  
## 4 0.6201497 0.6251909
```

Predictions using QDA model:

```
qda.class <- predict(qda.fit, test)$class
```

Confusion matrix:

```
table(qda.class, test.class)
```

```
##           test.class  
## qda.class  1  2  3  4  
##           1 90 28  1  2  
##           2 12 96 20  2  
##           3  0  5 75  5  
##           4 24  9 25 106
```

Rate of correct classifications:

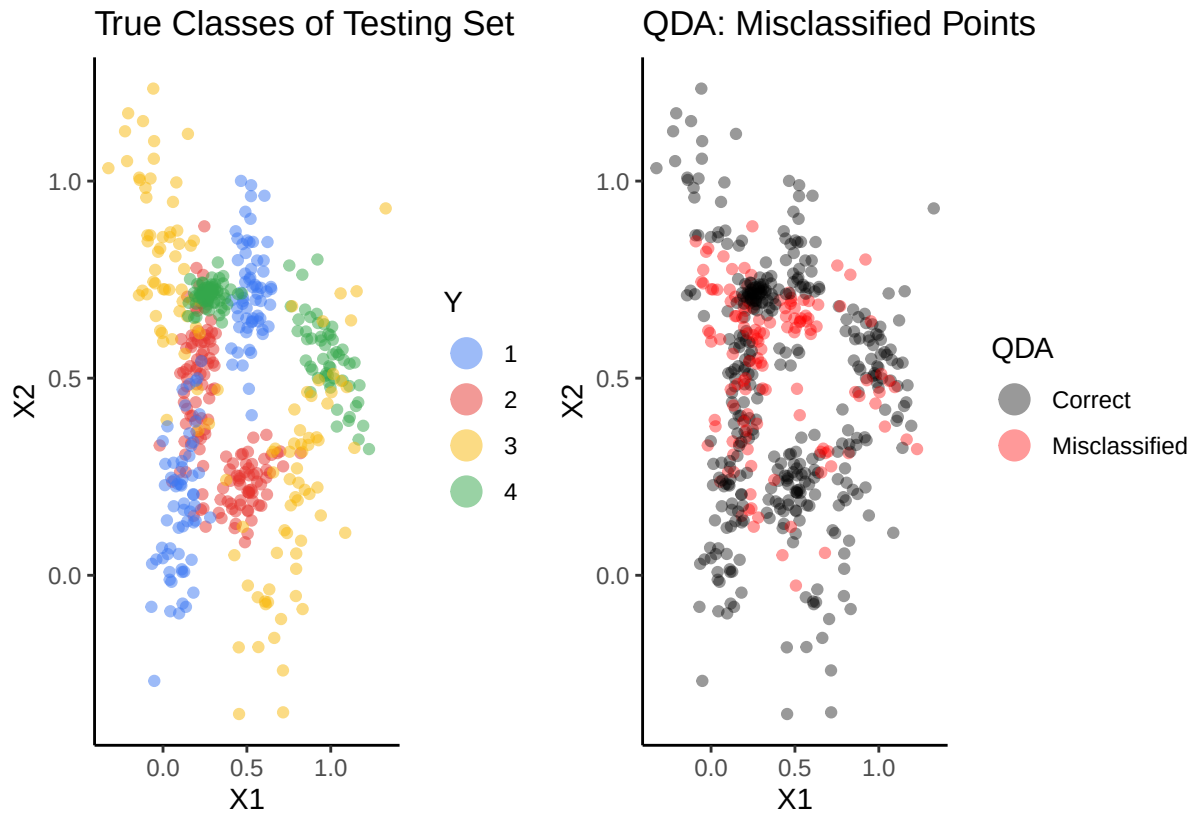
```
mean(qda.class == test.class)
```

```
## [1] 0.734
```

Test error:

```
mean(qda.class != test.class)
```

```
## [1] 0.266
```



4.

```
# Naive Bayes Model:
nb.fit <- naiveBayes(
  Y ~ X1 + X2,
  data = data2,
  subset = train
)
nb.fit

##
## Naive Bayes Classifier for Discrete Predictors
##
## Call:
## naiveBayes.default(x = X, y = Y, laplace = laplace)
##
## A-priori probabilities:
## Y
##    1    2    3    4
## 0.276 0.256 0.266 0.202
##
## Conditional probabilities:
##   X1
## Y    [,1]    [,2]
## 1 0.3290375 0.2207616
## 2 0.3507495 0.1940703
```



```

##      3 0.4526473 0.4335252
##      4 0.6201497 0.3683821
##
##      X2
## Y      [,1]      [,2]
##      1 0.4882679 0.3001793
##      2 0.3612605 0.1656382
##      3 0.4717941 0.3591727
##      4 0.6251909 0.1080046

# Predictions using Naive Bayes:
nb.class <- predict(nb.fit, test)

# Confusion matrix:
table(nb.class, test.class)

##           test.class
## nb.class    1    2    3    4
##           1 31 36 33 54
##           2 44 100 10 0
##           3 9 2 64 9
##           4 42 0 14 52

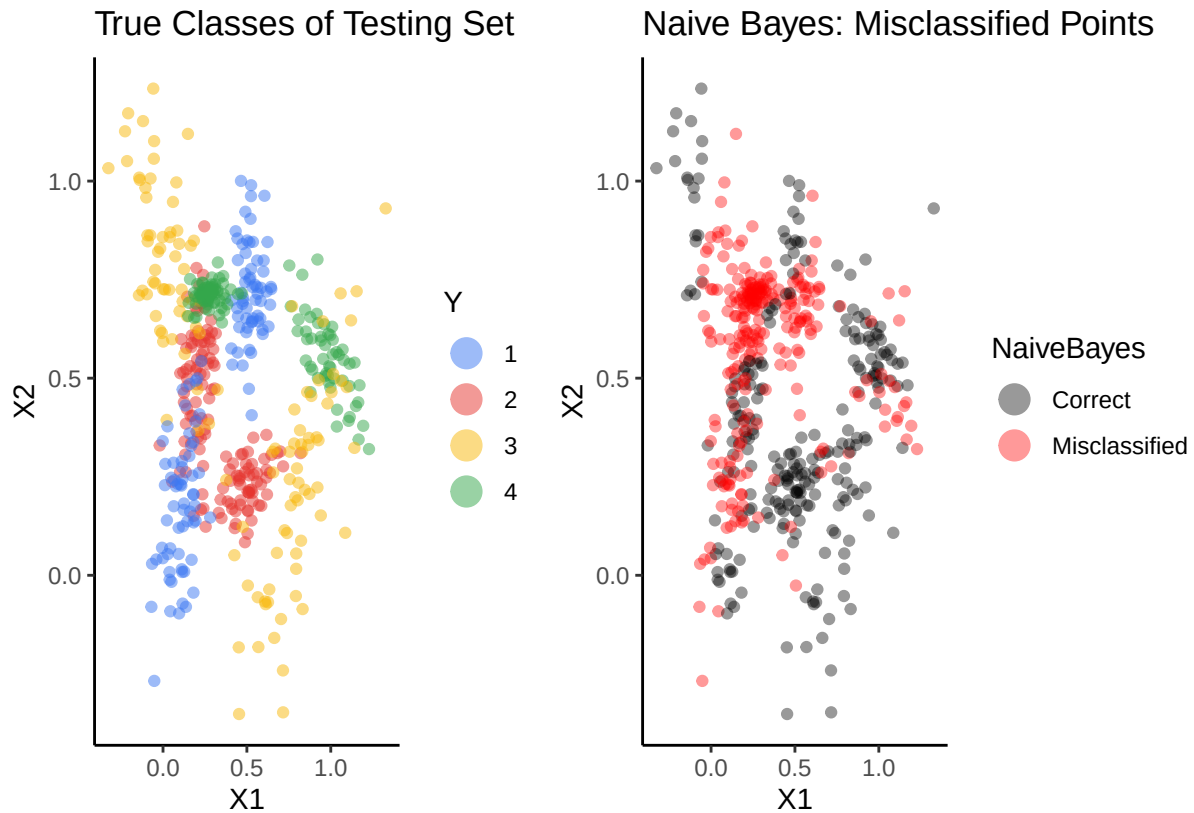
# Rate of correct classifications:
mean(nb.class == test.class)

## [1] 0.494

# Test error:
mean(nb.class != test.class)

## [1] 0.506

```



5.

```
# Standardizing X1 & X2:
standardized.X <- scale(data2[,c(1:2)])
```

```
# Updating training & testing sets:
train.X <- standardized.X[train, ]
test.X <- standardized.X[!train, ]
train.class <- data2$Y[train]
test.class <- data2$Y[!train]
```

```
# Function to find the best k for KNN:
best.k <- function(train.X, test.X, train.class, test.class, k.vals = seq(1, 100, by = 1)) {
  accuracy <- sapply(k.vals, function(k) {
    set.seed(10)
    mean(knn(train.X, test.X, train.class, k = k) == test.class)
  })
  best.k <- k.vals[which.max(accuracy)]
  return(best.k)
}
```

```
# Results:
best.k(train.X, test.X, train.class, test.class)
```

```
## [1] 13
```

```
# Predictions using KNN with k = 13:
set.seed(10)
knn.pred <- knn(train.X, test.X, train.class, k = 13)
```

```
# Confusion Matrix:
table(knn.pred, test.class)
```

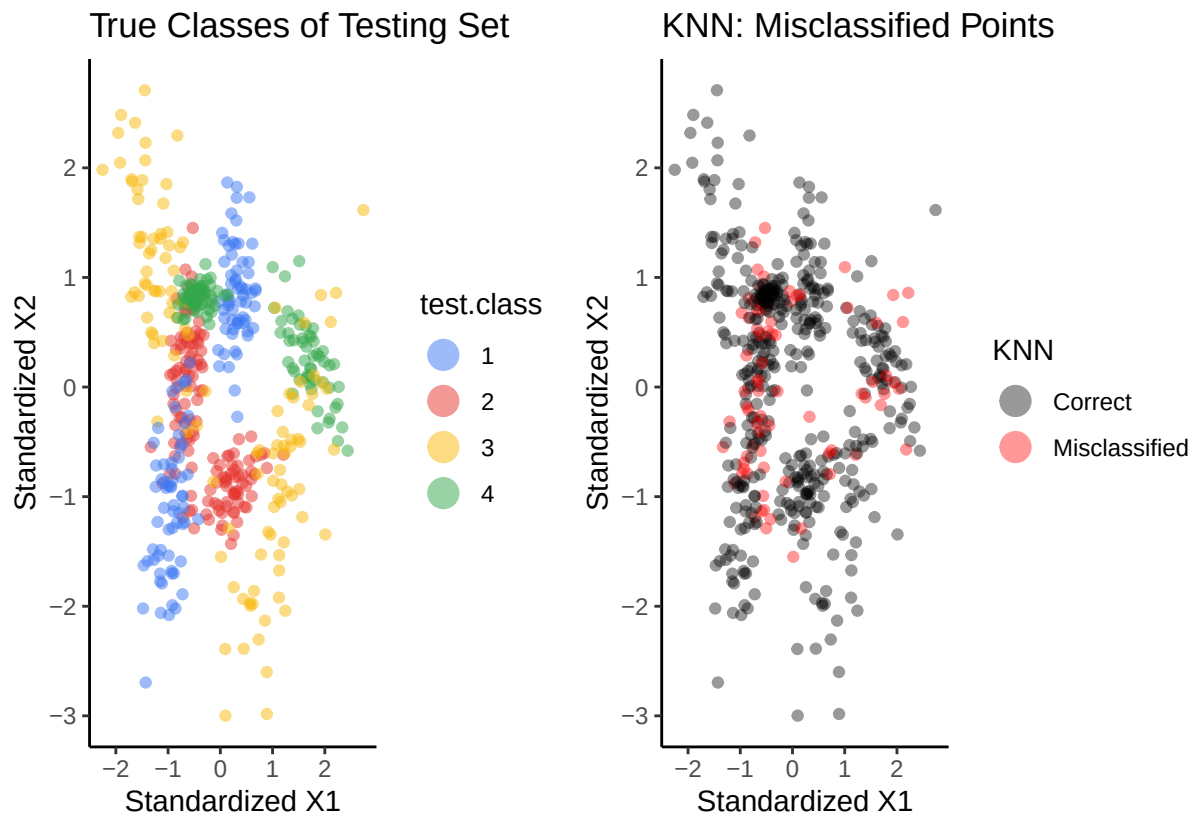
```
##          test.class
## knn.pred  1    2    3    4
##          1 114  14    1    4
##          2  11 110  15    0
##          3   0   2  88    0
##          4   1  12  17 111
```

```
# Rate of correct classifications:
mean(knn.pred == test.class)
```

```
## [1] 0.846
```

```
# Test Error:
mean(knn.pred != test.class)
```

```
## [1] 0.154
```



6.

Comparing the three testing procedures, the results are as follows:

Table 2: Test Error Comparisons

Method	Test_Error
QDA	26.6%
Naive Bayes	50.6%
KNN	15.4%

For this data, KNN performed better than QDA and Naive Bayes.